

UNIVERSIDADE CATÓLICA DO SALVADOR
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS

NEXUS

Documento de Arquitetura de Software

Plataforma inteligente de apoio ao levantamento, especificação, prototipação e suporte pós-entrega de soluções de software

Stack: Django · Django REST Framework · Celery · PostgreSQL · Angular

Autor: Tauan
Orientador(a): a definir

Salvador – Bahia
Outubro de 2026

Sumário

- 1 Objetivo e escopo do documento
- 2 Visão geral da solução
- 3 Estilo arquitetural adotado
- 4 Decisões arquiteturais (ADR)
- 5 Visão de containers
- 6 Organização em módulos de negócio
- 7 Camadas internas de cada módulo
- 8 Estrutura de diretórios
- 9 Ciclo de vida dos artefatos
- 10 Processamento assíncrono e contrato da API
- 11 Integração com Inteligência Artificial
- 12 Persistência e modelo de dados
- 13 Segurança
- 14 Atendimento aos requisitos não funcionais
- 15 Implantação e ambientes
- 16 Estratégia de testes
- 17 Riscos arquiteturais e mitigações
- 18 Evolução prevista

1 Objetivo e escopo do documento

Este documento descreve a arquitetura de software do Nexus, plataforma proposta como Trabalho de Conclusão de Curso. Ele registra as decisões estruturais tomadas, as alternativas avaliadas e as razões de cada escolha, de modo que a implementação possa ser conduzida sem reabrir discussões já resolvidas e que a banca possa avaliar o raciocínio técnico, e não apenas o resultado.

O documento cobre a organização do código, a divisão de responsabilidades, o fluxo de processamento assistido por Inteligência Artificial, a estratégia de segurança e o plano de implantação. Não cobre a especificação funcional da plataforma, que consta da proposta de TCC, nem o detalhamento de interface, tratado nos protótipos.

2 Visão geral da solução

O Nexus é uma aplicação web que apoia o ciclo de vida de uma demanda de tecnologia: recebe a necessidade por transcrição de reunião ou por entrevista guiada em chat, estrutura o levantamento, gera requisitos assistidos por IA, recomenda o cenário técnico mais aderente, produz um protótipo estático de telas, organiza o plano de trabalho conforme a metodologia escolhida pelo desenvolvedor e, após a entrega, concentra os chamados abertos de dentro do próprio produto construído.

Três características do problema determinam a arquitetura:

- Operações longas. Transcrever uma reunião ou gerar um conjunto de requisitos leva dezenas de segundos a alguns minutos, o que torna inviável o processamento dentro do ciclo de requisição e resposta HTTP.
- Resultado não determinístico e sujeito a revisão. Nenhum artefato produzido pela IA vale como decisão antes de passar por validação humana, o que exige versionamento, estados explícitos e trilha de auditoria.
- Dependência de serviço externo instável e tarifado. O provedor de modelo de linguagem pode falhar, mudar de versão ou ser substituído, o que exige isolamento por interface.

3 Estilo arquitetural adotado

Decisão central

O Nexus é implementado como um monólito modular em Django, com fronteiras internas explícitas entre módulos de negócio, processamento assíncrono por fila e front-end desacoplado em Angular consumindo uma API REST.

A expressão “monólito modular” descreve um sistema implantado como uma única aplicação, porém internamente dividido em módulos com responsabilidade própria, que se comunicam por interfaces definidas e não por acesso direto às tabelas uns dos outros. É um meio-termo deliberado entre o monólito tradicional, em que tudo se mistura, e a arquitetura de microserviços, em que cada módulo vira um processo independente.

Comparação das alternativas consideradas:

Alternativa	Vantagem	Por que foi descartada ou adotada
Monólito tradicional	Implementação mais rápida no início.	Descartado: sem fronteiras internas, a regra de negócio se espalha entre views e models, e o trabalho perde o conteúdo de engenharia de software que a banca avalia.
Microserviços	Escalabilidade e implantação independentes por serviço.	Descartado: exige orquestração, comunicação entre serviços, consistência distribuída e observabilidade. O custo operacional não se justifica para um TCC com um desenvolvedor, e desloca o esforço da contribuição real do trabalho.
Monólito modular	Fronteiras internas claras com uma única unidade de implantação.	Adotado: oferece a separação necessária para a qualidade do projeto, mantém a operação simples e permite extrair um módulo para serviço próprio no futuro, se houver necessidade real.

A decisão de separar o front-end em uma aplicação Angular independente, em vez de usar os templates do Django, decorre da natureza da interface: edição de requisitos item a item, acompanhamento de tarefas em segundo plano, quadro de planejamento e visualização de protótipo são interações com estado em cliente, mal atendidas por páginas renderizadas no servidor.

4 Decisões arquiteturais (ADR)

Registro resumido das decisões, no formato de Architecture Decision Records. Cada decisão indica o que foi escolhido, o que foi preterido e qual consequência prática foi aceita.

ID	Decisão	Alternativa preterida	Consequência aceita
ADR-01	Monólito modular em Django.	Microserviços.	Escala vertical; separação depende de disciplina de código, não de processo.
ADR-02	Front-end Angular desacoplado, consumindo API REST.	Templates Django com renderização no servidor.	Dois projetos para manter e necessidade de tratar CORS e autenticação por token.
ADR-03	Toda chamada à IA é assíncrona, via Celery e Redis.	Chamada síncrona dentro da requisição HTTP.	Complexidade de fila e de acompanhamento de estado da tarefa.
ADR-04	Acesso ao modelo de linguagem por uma porta LLMPProvider, com adaptadores.	Chamada direta ao SDK do provedor dentro do serviço.	Uma camada de indireção a mais, em troca de troca de provedor sem reescrita.
ADR-05	Busca vetorial no próprio PostgreSQL, com a extensão pgvector.	Banco vetorial dedicado (Pinecone, Qdrant, Weaviate).	Desempenho inferior em escala muito alta, irrelevante no volume previsto; evita mais um serviço.
ADR-06	Auditoria em tabela append-only, sem atualização nem exclusão.	Histórico derivado de campos de data no próprio registro.	Crescimento da tabela de eventos; em troca, rastreabilidade íntegra das

ID	Decisão	Alternativa preterida	Consequência aceita
			validações.
ADR-07	Acompanhamento de tarefas por polling HTTP.	WebSocket ou Server-Sent Events.	Latência de alguns segundos na atualização; em troca, infraestrutura e código muito mais simples.
ADR-08	Regra de negócio isolada em camada de domínio sem dependência de Django.	Regra de negócio em métodos dos models.	Mais arquivos por módulo; em troca, testes sem banco e independência de framework.

5 Visão de containers

A Figura 1 apresenta os componentes executáveis do sistema e a comunicação entre eles, no nível de containers do modelo C4. Cada bloco corresponde a um processo ou serviço que roda separadamente.

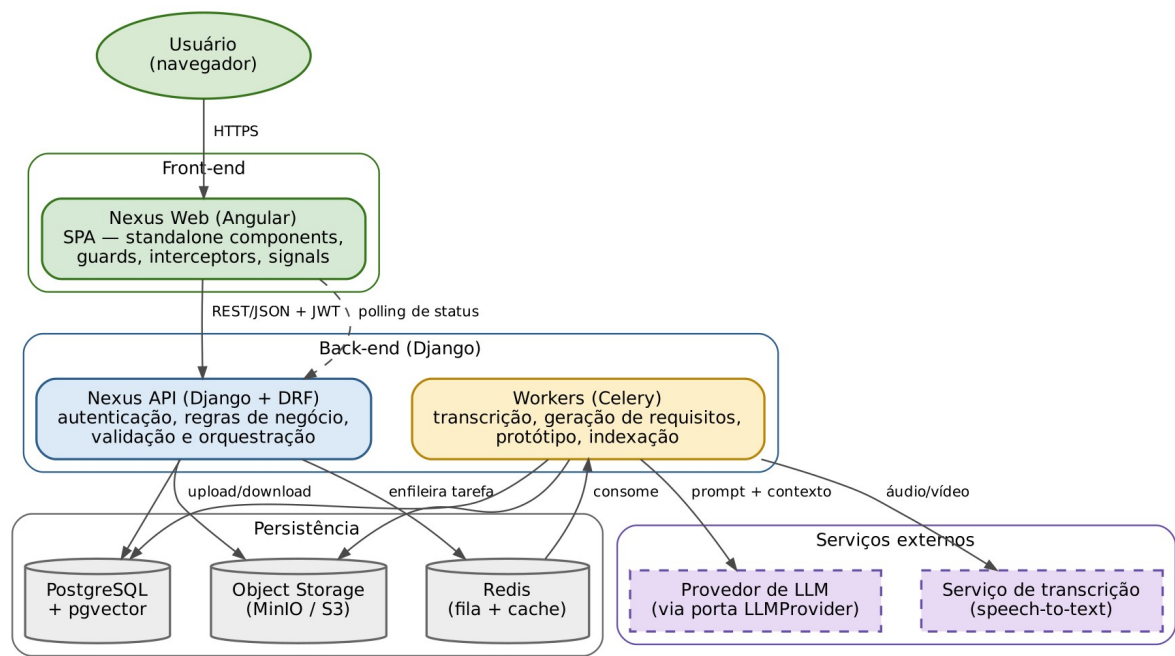


Figura 1 – Visão de containers do Nexus

Container	Tecnologia	Responsabilidade
Nexus Web	Angular	Interface do usuário, controle de sessão, acompanhamento de tarefas e renderização do protótipo gerado.
Nexus API	Django + Django REST Framework	Autenticação, autorização, validação de entrada, execução das regras de negócio síncronas e enfileiramento de tarefas longas.
Workers	Celery	Transcrição, geração de levantamento e requisitos, pontuação de cenário, geração de protótipo e indexação da base de conhecimento.
PostgreSQL	PostgreSQL + pgvector	Dados relacionais, versionamento de artefatos, trilha de auditoria e índice vetorial da base de conhecimento.
Redis	Redis	Fila de tarefas do Celery, cache de respostas e controle de limite de requisições.
Object Storage	MinIO em desenvolvimento, S3 em produção	Arquivos de transcrição, áudio, vídeo, anexos e protótipos gerados.
Provedor de LLM	Serviço externo	Geração de texto estruturado a partir do contexto enviado.
Serviço de transcrição	Serviço externo	Conversão de áudio e vídeo em texto.

A API e os workers compartilham o mesmo código-fonte e o mesmo banco: são o mesmo projeto Django iniciado com comandos diferentes. Isso mantém a regra de negócio em um só lugar, independentemente de ser executada de forma síncrona ou em segundo plano.

6 Organização em módulos de negócio

Os aplicativos Django são divididos por contexto de negócio, e não por camada técnica. Não existe um aplicativo chamado “models” ou “views”: cada aplicativo representa uma etapa do processo descrito na proposta, o que mantém a correspondência direta entre a documentação funcional e o código.

Módulo (app)	Responsabilidade	Entidades principais
core	Base compartilhada: modelo-base com identificador e datas, exceções, paginação, trilha de auditoria.	EventoAuditoria
contas	Usuários, perfis, permissões e atribuição de responsáveis por validação.	Usuario, Perfil, Atribuicao
demandas	Registro e acompanhamento das necessidades.	Demanda, Anexo
levantamento	Processamento de transcrições e condução da entrevista guiada adaptativa.	FonteInformacao, SessaoEntrevista, Levantamento
requisitos	Requisitos funcionais, não funcionais, regras e critérios de aceitação, com versionamento.	Requisito, VersaoRequisito, Baseline
validacao	Fila de aprovação, decisões e histórico de revisões.	SolicitacaoValidacao, Decisao
cenario	Extração de sinais e pontuação de aderência dos cenários técnicos.	AnaliseCenario, Sinal, Recomendacao
prototipo	Geração, versionamento e comentários do protótipo estático.	Prototipo, Tela, Comentario
planejamento	Metodologia escolhida e artefatos correspondentes.	PlanoTrabalho, ItemBacklog, Iteracao
produtos	Soluções publicadas, versões e vínculo com o projeto de origem.	Produto, Versao, Publicacao
chamados	Abertura, triagem assistida, atendimento e conversão em nova demanda.	Chamado, Triagem, Atendimento
conhecimento	Procedimentos validados, indexação vetorial e reuso.	ItemConhecimento, Trecho, Embedding
ia	Orquestração das chamadas ao modelo, prompts versionados e registro de execuções.	Job, PromptTemplate, LogIA

Regra de fronteira entre módulos

Um módulo nunca importa models de outro módulo diretamente. A comunicação ocorre por meio dos serviços expostos pelo módulo de destino ou por eventos de domínio. Essa é a

disciplina que separa um monólito modular de um monólito comum, e é o que permitiria, no futuro, extrair um módulo para um serviço próprio sem reescrever o sistema.

7 Camadas internas de cada módulo

Dentro de cada módulo, o código é organizado em quatro camadas, com direção de dependência bem definida, conforme a Figura 2. A camada de domínio é o centro do desenho: ela não conhece o Django, não conhece o banco e não conhece o provedor de IA.

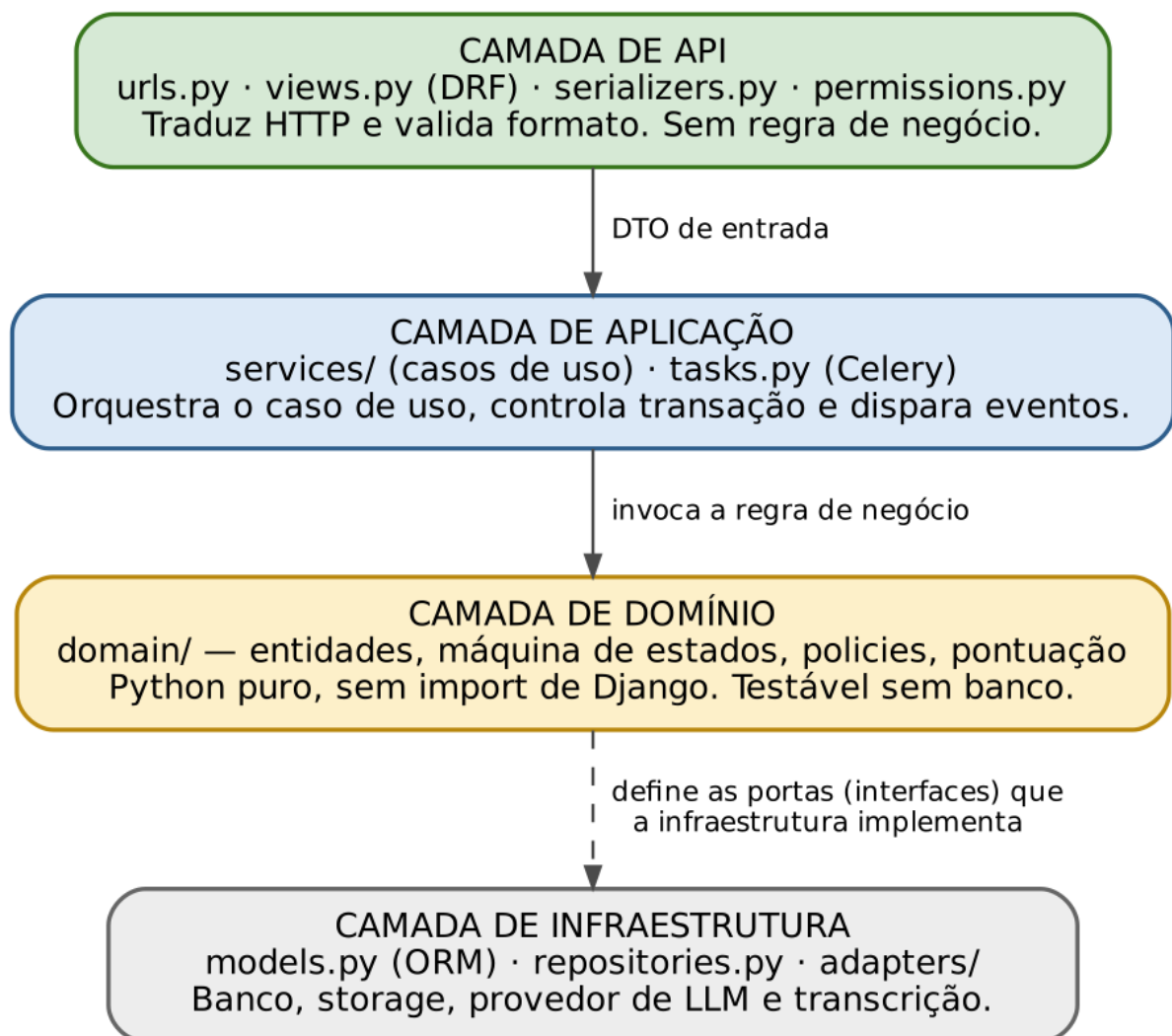


Figura 2 – Camadas internas de um módulo e direção das dependências

Camada	Conteúdo	O que nunca deve conter
API	Rotas, views do DRF, serializers, classes de permissão.	Regra de negócio, consulta complexa ao banco, chamada a serviço externo.
Aplicação	Casos de uso em services/, tarefas Celery em tasks.py, controle de transação.	Lógica de decisão de negócio que pertença ao domínio; detalhes de HTTP.

Camada	Conteúdo	O que nunca deve conter
Domínio	Entidades, máquina de estados, políticas de permissão de negócio, cálculo de pontuação de cenário, interfaces (portas).	Qualquer import de Django, do ORM, de biblioteca de rede ou de SDK externo.
Infraestrutura	Models do ORM, repositórios, adaptadores de LLM, transcrição e armazenamento.	Decisão de negócio; orquestração de caso de uso.

A camada de aplicação é a única que conversa com as três outras: recebe o pedido da API, consulta a infraestrutura para carregar dados, submete a decisão ao domínio e grava o resultado. O domínio define interfaces — por exemplo, um repositório de requisitos ou um provedor de linguagem — e a infraestrutura as implementa, o que caracteriza a inversão de dependência.

A justificativa prática é direta: a máquina de estados do artefato, as regras de quem pode validar o quê e o cálculo de aderência de cenário são testáveis sem subir banco, sem subir servidor e sem gastar crédito de API. Esses testes são rápidos e determinísticos, o que viabiliza a execução repetida dos casos de teste previstos na avaliação do trabalho.

8 Estrutura de diretórios

8.1 Back-end

```
nexus-api/
├── config/
│   ├── settings/
│   │   ├── base.py          # configuração comum
│   │   ├── dev.py           # desenvolvimento
│   │   └── prod.py          # produção
│   ├── urls.py              # roteamento raiz, versionado em /api/v1/
│   ├── celery.py            # instância e descoberta de tarefas
│   ├── asgi.py
│   └── wsgi.py
├── apps/
│   ├── core/                # base compartilhada e auditoria
│   ├── contas/
│   ├── demandas/
│   ├── levantamento/
│   ├── requisitos/
│   ├── validacao/
│   ├── cenario/
│   ├── prototipo/
│   ├── planejamento/
│   ├── produtos/
│   ├── chamados/
│   ├── conhecimento/
│   └── ia/
├── requirements/
│   ├── base.txt
│   ├── dev.txt
│   └── prod.txt
├── docker-compose.yml
├── Dockerfile
├── .env.example
└── manage.py
```

Organização interna de um módulo, usando requisitos como exemplo:

```
apps/requisitos/
├── api/
│   ├── urls.py
│   ├── views.py             # ViewSets do DRF – apenas tradução HTTP
│   ├── serializers.py       # campos explícitos, nunca "__all__"
│   └── permissions.py       # permissão por objeto
├── services/
│   ├── gerar_requisitos.py   # caso de uso: dispara geração assistida
│   ├── aprovar_requisito.py  # caso de uso: aplica decisão de validação
│   └── fechar_baseline.py
├── domain/
│   ├── entities.py          # Requisito como objeto de domínio
│   ├── estados.py           # máquina de estados e transições válidas
│   ├── policies.py          # quem pode validar o quê
│   └── ports.py              # interfaces que a infraestrutura implementa
├── models.py                 # mapeamento ORM
├── repositories.py           # consultas isoladas do caso de uso
├── tasks.py                  # tarefas Celery
├── admin.py
├── migrations/
├── tests/
│   └── test_dominio.py       # rápido, sem banco
```

```
└─ test_services.py
└─ test_api.py
```

8.2 Front-end

```
nexus-web/
└─ src/app/
  └─ core/
    └─ auth/          # serviço de sessão, guard, renovação de token
    └─ http/          # interceptors: token, erro, correlação
    └─ config/        # ambiente e constantes
  └─ shared/
    └─ ui/            # componentes reutilizáveis
    └─ pipes/
    └─ models/        # tipos espelhando os contratos da API
  └─ features/        # um módulo por contexto, carregado sob demanda
    └─ demandas/
      └─ pages/
      └─ components/
      └─ data/        # serviços HTTP e estado da feature
    └─ levantamento/
    └─ requisitos/
    └─ validacao/
    └─ cenario/
    └─ prototipo/
    └─ planejamento/
    └─ produtos/
    └─ chamados/
    └─ conhecimento/
  └─ app.routes.ts    # rotas com carregamento sob demanda
  └─ app.config.ts
└─ src/environments/
└─ Dockerfile
```

O front-end utiliza componentes independentes (standalone components), rotas com carregamento sob demanda por feature e sinais (signals) para estado local, dispensando bibliotecas externas de gerenciamento de estado. A correspondência entre as pastas de features e os módulos do back-end é intencional: facilita a navegação e evita divergência de vocabulário entre as duas bases de código.

9 Ciclo de vida dos artefatos

Todo artefato produzido com apoio da IA — levantamento, requisito, recomendação de cenário, protótipo, procedimento de conhecimento — percorre a mesma máquina de estados, implementada uma única vez no módulo core e reutilizada pelos demais. É esse mecanismo que materializa, no código, o princípio de que a IA propõe e o humano decide.

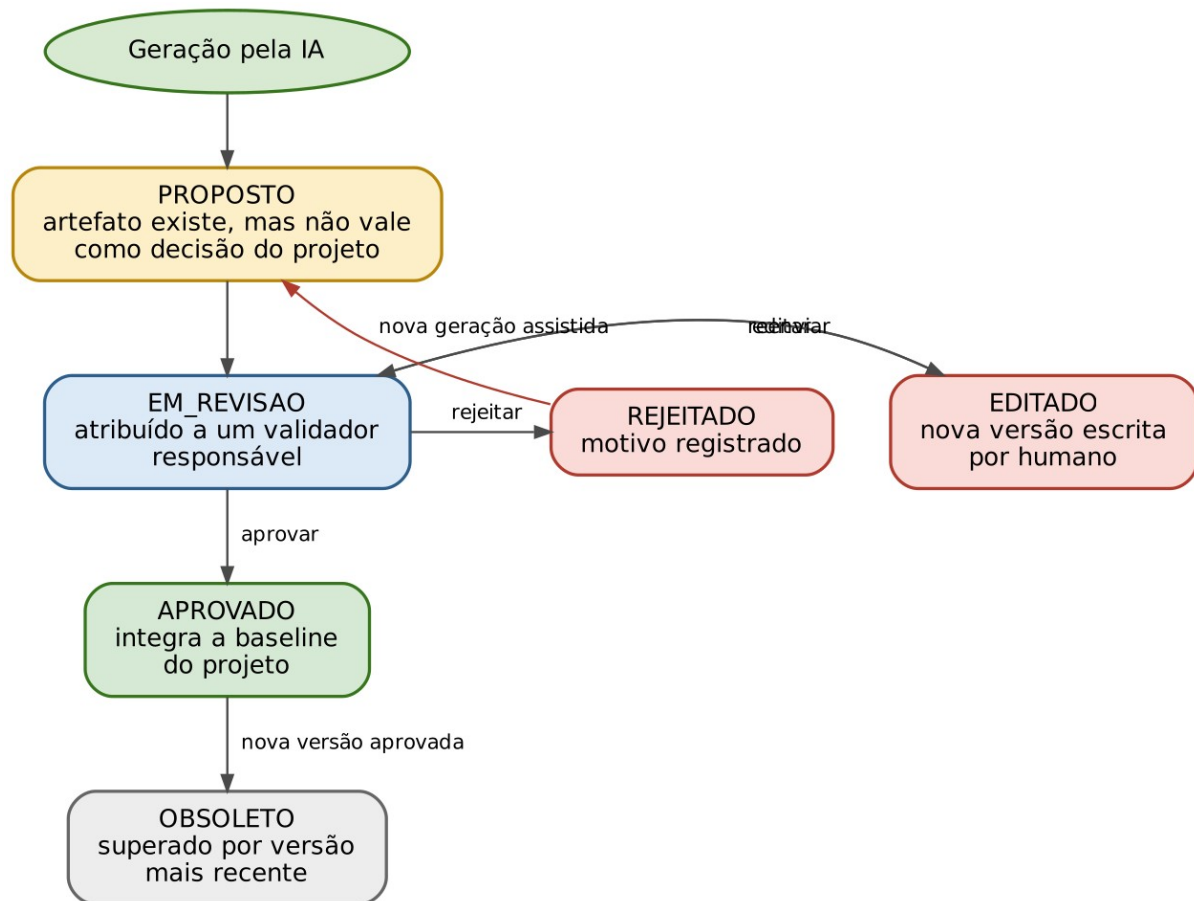


Figura 3 – Máquina de estados comum a todos os artefatos assistidos por IA

Transição	Quem pode executar	Efeito registrado
Geração → PROPOSTO	Sistema (worker).	Artefato criado com referência ao job, ao modelo utilizado e à origem da informação.
PROPOSTO → EM_REVISAO	Sistema, ao atribuir validador.	Responsável designado e prazo de revisão.
EM_REVISAO → APROVADO	Validador designado para o tipo de artefato.	Evento de aprovação com autor, data e versão aprovada.
EM_REVISAO → EDITADO	Validador designado.	Nova versão com autoria humana; versão anterior preservada.
EM_REVISAO → REJEITADO	Validador designado.	Motivo da rejeição, obrigatório, para análise posterior de qualidade.
EDITADO → EM_REVISAO	Autor da edição.	Reenvio para nova apreciação.

Transição	Quem pode executar	Efeito registrado
REJEITADO → PROPOSTO	Sistema, em nova geração assistida.	Nova tentativa vinculada ao motivo da rejeição anterior.
APROVADO → OBSOLETO	Sistema, ao aprovar versão posterior.	Encadeamento entre versões; a baseline aponta sempre para a versão vigente.

Por que isso importa na avaliação do trabalho

A taxa de aprovação sem edição, a taxa de rejeição e os motivos registrados são extraídos diretamente desta máquina de estados. As métricas previstas na avaliação do TCC não dependem de instrumentação adicional: elas são consultas sobre os eventos que o próprio fluxo de uso produz.

10 Processamento assíncrono e contrato da API

Operações que dependem do provedor de IA ou de transcrição são sempre executadas fora do ciclo de requisição e resposta. A Figura 4 detalha o percurso completo de uma geração de requisitos, do clique do usuário ao registro da decisão de validação.

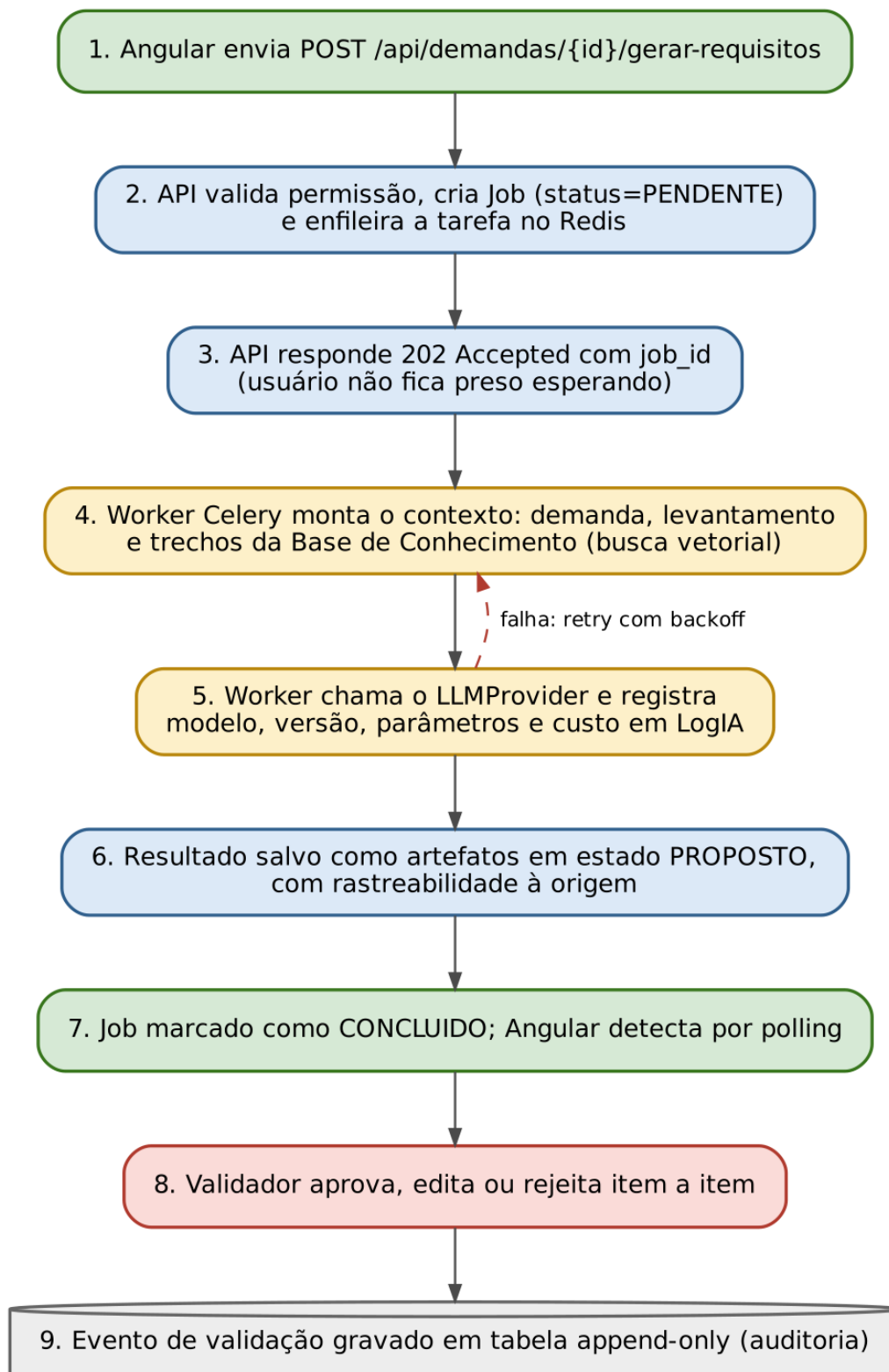


Figura 4 – Fluxo de uma geração assistida, do pedido à validação

Padrão de contrato adotado para toda operação longa:

Operação	Método e rota	Resposta
Solicitar geração	POST /api/v1/demandas/{id}/requisitos/gera	202 Accepted com job_id e situação inicial.

Operação	Método e rota	Resposta
	r	
Consultar situação	GET /api/v1/jobs/{job_id}	PENDENTE, EM_EXECUCAO, CONCLUIDO ou FALHOU, com percentual quando aplicável.
Obter resultado	GET /api/v1/demandas/{id}/requisitos	Lista de artefatos em estado PROPOSTO, com origem e nível de confiança.
Decidir validação	POST /api/v1/requisitos/{id}/decisao	Artefato atualizado e evento de auditoria gravado.

O front-end consulta a situação do job em intervalo crescente, começando em dois segundos. Ao receber CONCLUIDO, busca o resultado. A escolha por polling, em vez de WebSocket, é deliberada: elimina a necessidade de camada ASGI com canais, de gerenciamento de conexões persistentes e de reconexão, a um custo de latência irrelevante para uma operação que já leva dezenas de segundos.

Tarefas definem número máximo de tentativas, intervalo crescente entre tentativas e limite de tempo. Toda falha é registrada no job com mensagem tratada, nunca com a exceção bruta devolvida ao usuário.

11 Integração com Inteligência Artificial

11.1 Porta e adaptadores

O domínio define a interface `LLMProvider`, com operações de geração de texto estruturado e de cálculo de representação vetorial. A infraestrutura fornece implementações concretas para os provedores comerciais e uma implementação falsa, de resposta fixa, utilizada nos testes automatizados.

```
# apps/ia/domain/ports.py — sem dependência de framework
class LLMProvider(Protocol):
    def gerar_estruturado(
        self, prompt: str, schema: dict, contexto: list[str]
    ) -> ResultadoGeracao: ...

    def embedding(self, texto: str) -> list[float]: ...

# apps/ia/adapters/
# openai_adapter.py      provedor comercial
# anthropic_adapter.py   provedor comercial alternativo
# fake_adapter.py        resposta fixa, usada nos testes
```

O adaptador falso é um elemento central da estratégia de avaliação: permite executar os casos de teste quantas vezes forem necessárias, com resultado idêntico, sem custo e sem depender de conexão. A troca de implementação é feita por configuração, sem alteração de código de negócio.

11.2 Respostas estruturadas e validação

O modelo nunca é consultado em texto livre quando o resultado alimenta um artefato. Toda geração exige um esquema JSON, e a resposta é validada contra esse esquema antes de ser persistida. Resposta fora do formato é tratada como falha da tarefa, com nova tentativa, e não como conteúdo válido. Esse cuidado atende diretamente ao risco de consumo inseguro de serviços externos.

11.3 Recuperação de contexto

A geração é fundamentada em conteúdo do próprio projeto, por meio de geração aumentada por recuperação. Os itens da Base de Conhecimento e os artefatos já aprovados são fragmentados, convertidos em vetores e indexados no PostgreSQL com a extensão `pgvector`. Antes de cada geração, o sistema recupera os fragmentos mais próximos da demanda em questão e os inclui no contexto enviado ao modelo.

A opção por `pgvector`, em vez de um banco vetorial dedicado, reduz em um o número de serviços a operar e mantém o conteúdo e seu índice na mesma transação. No volume previsto para o trabalho — milhares de fragmentos, não milhões — a diferença de desempenho não é perceptível.

11.4 Registro de execuções

Cada chamada ao provedor grava um registro com modelo, versão, identificador do template de prompt, parâmetros, quantidade de tokens, custo estimado, duração e identificador do artefato gerado. Esse registro sustenta a auditoria exigida pelos requisitos não funcionais, permite estimar custo operacional e fornece dados para a análise de resultados do trabalho.

Os prompts ficam versionados como templates no banco, e não embutidos no código. Alterar um prompt passa a ser uma mudança rastreável, o que é indispensável quando se pretende comparar a qualidade das gerações ao longo do desenvolvimento.

12 Persistência e modelo de dados

O PostgreSQL concentra os dados relacionais, o versionamento dos artefatos, a trilha de auditoria e o índice vetorial. Convenções adotadas:

- Identificador primário do tipo UUID em todas as entidades, evitando exposição de sequência numérica nas rotas da API.
- Modelo-base no módulo core com identificador, data de criação, data de atualização e autor da última alteração.
- Versionamento por encadeamento: cada nova versão referencia a anterior, e o registro vigente é identificado por sinalizador próprio.
- Exclusão lógica para artefatos de projeto; exclusão física apenas para dados temporários de processamento.
- Tabela de eventos de auditoria em modo somente inclusão, sem permissão de atualização ou exclusão pela aplicação.
- Armazenamento de arquivos fora do banco, com o banco guardando apenas a referência ao objeto.

13 Segurança

A plataforma trata conteúdo sensível: transcrições de reuniões corporativas, regras de negócio e documentação técnica de sistemas internos. A arquitetura incorpora controles em todas as camadas, organizados a seguir conforme as categorias do OWASP API Security Top 10.

Categoria OWASP	Risco no contexto do Nexus	Controle arquitetural
Autorização em nível de objeto	Usuário acessar demanda ou requisito de projeto do qual não participa.	Permissão por objeto em todas as rotas; consultas sempre filtradas pelo vínculo do usuário, nunca apenas pelo identificador recebido.
Autenticação quebrada	Roubo ou reuso de token.	JWT de curta duração, token de renovação rotativo com lista de revogação, invalidação de sessão no logout.
Autorização em nível de propriedade	Alteração de campos que o perfil não deveria editar, como situação de validação.	Serializers com campos explícitos e campos somente leitura; transição de estado apenas por caso de uso próprio.
Consumo irrestrito de recursos	Uso abusivo das rotas de IA, com impacto financeiro direto.	Limite de requisições por usuário e por projeto, cota de gerações por demanda e limite de tamanho de arquivo enviado.
Autorização em nível de função	Perfil de solicitante executando ação de responsável técnico.	Classes de permissão por perfil combinadas com políticas de negócio na camada de domínio.
Configuração incorreta	Exposição de dados em ambiente de produção.	Configuração separada por ambiente, segredos em variáveis de ambiente, depuração desativada, cabeçalhos de segurança e CORS restrito à origem conhecida.
Inventário inadequado	Rotas antigas acessíveis sem controle.	Versionamento da API em /api/v1/ e documentação gerada automaticamente em OpenAPI.
Consumo inseguro de serviço externo	Resposta do modelo tratada como confiável.	Validação da resposta contra esquema JSON, tempo limite, número máximo de tentativas e tratamento de falha como erro de tarefa.

13.1 Proteção de dados pessoais

Antes de qualquer envio ao provedor externo, as transcrições passam por rotina de anonimização que substitui nomes próprios, endereços de correio eletrônico, números de telefone e identificadores pessoais por marcadores. O conteúdo original permanece no ambiente da plataforma; o conteúdo que sai é a versão anonimizada. Essa separação é registrada no log de execução e é o controle que sustenta a conformidade pretendida com a Lei Geral de Proteção de Dados.

13.2 Trilha de auditoria

Toda decisão de validação, publicação de produto e alteração de conteúdo da Base de Conhecimento gera um evento imutável com autor, data, ação, artefato e versão. A aplicação não possui caminho de código capaz de atualizar ou excluir esses eventos. É esse registro que permite responder, meses depois, quem aprovou determinado requisito e com base em qual versão.

14 Atendimento aos requisitos não funcionais

Rastreabilidade entre os requisitos não funcionais declarados na proposta e os mecanismos arquiteturais que os atendem.

RNF	Requisito	Mecanismo arquitetural
RNF0 1	Autenticação e controle de acesso por perfil.	JWT com renovação rotativa; permissões por perfil e por objeto no DRF.
RNF0 2	Auditoria de aprovações, edições e rejeições.	Tabela de eventos somente inclusão no módulo core.
RNF0 3	Resultados da IA sempre apresentados como sugestão.	Estado PROPOSTO na máquina de estados; o front-end identifica visualmente a origem.
RNF0 4	Interface responsiva.	Angular com layout responsivo e componentes independentes.
RNF0 5	Proteção dos dados armazenados e em trânsito.	HTTPS obrigatório, segredos em variáveis de ambiente, arquivos em armazenamento com acesso assinado.
RNF0 6	Anonimização antes do envio ao modelo.	Rotina de anonimização obrigatória no adaptador, anterior à chamada externa.
RNF0 7	Substituição do provedor de IA sem reescrita.	Porta LLMProvider com adaptadores intercambiáveis por configuração.
RNF0 8	Tempo de resposta adequado e tarefas longas em segundo plano.	Celery com Redis; respostas 202 e acompanhamento por job.
RNF0 9	Nenhuma ação crítica executada automaticamente.	Ausência de transição automática para APROVADO; execução de procedimento sempre precedida de validação humana.
RNF1 0	Registro de modelo, versão e parâmetros de cada geração.	Entidade LogIA e templates de prompt versionados.

15 Implantação e ambientes

O ambiente é definido por contêineres, o que padroniza a execução entre a máquina de desenvolvimento e o ambiente de demonstração e elimina divergências de configuração durante a avaliação do trabalho.

Serviço	Imagem ou base	Função
web	Imagem própria (Python)	Aplicação Django servida por Gunicorn.
worker	Mesma imagem de web	Processo Celery consumindo a fila.
beat	Mesma imagem de web	Agendador de tarefas periódicas, como reindexação e limpeza.
db	postgres com pgvector	Banco relacional e índice vetorial.
redis	redis	Fila de tarefas e cache.
minio	minio	Armazenamento de objetos em desenvolvimento.
front	Imagem própria (Node e Nginx)	Build do Angular servido por Nginx.
nginx	nginx	Proxy reverso, terminação TLS e roteamento entre front e API.

A configuração é lida de variáveis de ambiente, sem valores sensíveis no repositório. Há três perfis previstos: desenvolvimento, com recarga automática e provedor falso de IA; homologação, com provedor real e dados fictícios; e demonstração, utilizado na banca, com dados preparados dos casos de teste.

16 Estratégia de testes

Nível	Alvo	Características
Domínio	Máquina de estados, políticas de validação, pontuação de cenário.	Sem banco e sem rede; execução em segundos; maior cobertura esperada.
Serviços	Casos de uso completos com banco de teste e provedor falso de IA.	Verificam transação, efeitos colaterais e geração de eventos de auditoria.
API	Rotas do DRF: autenticação, permissão, formato de entrada e saída.	Incluem tentativas de acesso indevido a objetos de outro usuário.
Integração com IA	Contrato do adaptador contra resposta registrada do provedor.	Executados sob demanda, não em cada alteração, para controlar custo.
Aceitação	Casos de teste CT01 a CT10 definidos na proposta.	Executados na avaliação final, com coleta das métricas do trabalho.

A separação entre testes de domínio e os demais é o que torna viável executar a suíte com frequência: a maior parte das regras que sustentam o trabalho é verificada sem infraestrutura.

17 Riscos arquiteturais e mitigações

Risco	Impacto	Mitigação
Indisponibilidade do provedor de IA durante a apresentação.	Alto	Provedor falso configurável e resultados pré-gerados dos casos de teste armazenados no ambiente de demonstração.
Custo das chamadas ao modelo acima do previsto.	Médio	Cota por demanda, cache de respostas idênticas, uso do provedor falso em desenvolvimento e registro de custo por execução.
Vazamento da disciplina de módulos, com importações cruzadas.	Médio	Convenção documentada e verificação por ferramenta de análise estática na integração contínua.
Geração de protótipo estático mais complexa que o estimado.	Alto	Tratada como entregável de segunda prioridade; versão inicial limitada a um conjunto fixo de componentes.
Crescimento do índice vetorial degradando a consulta.	Baixo	Índice apropriado no pgvector e reindexação periódica; volume previsto está muito abaixo do limite prático.
Acúmulo de escopo além do prazo do TCC.	Alto	Implementação orientada pelo MVP definido na proposta; módulos adicionais declarados como trabalho futuro.

18 Evolução prevista

A arquitetura foi desenhada para permitir, sem reescrita, os seguintes passos posteriores à entrega do trabalho:

- Extração do módulo de IA para serviço próprio, caso o consumo justifique escala independente — viável porque o acesso já ocorre por interface.
- Substituição do polling por notificação em tempo real, com adição de camada ASGI, sem alteração das regras de negócio.
- Cache semântico de gerações, reaproveitando resultados de demandas suficientemente próximas e reduzindo custo.
- Aplicativo móvel consumindo a mesma API, já versionada e documentada em OpenAPI.
- Integração com repositórios de código e ferramentas de gestão de tarefas, por meio de novos adaptadores na camada de infraestrutura.